React Native Practical Programs

Shortened Code Edition

All 15 practicals combined into one PDF. Each program is condensed while preserving its original purpose, key functionality, and theme.

How to use

• These examples are React Native (not browser-based React.js) and use JavaScript.
• For each practical, copy only its code into `App.js`, except Practical 14 which also requires `firebaseConfig.js`.
• Install the packages shown in the relevant practical before running it.
• Start an Expo project with `npx create-expo-app@latest MyApp --template blank`, then run `npx expo start`.

1. Installation and Setup

Purpose: Create and run a blank Expo React Native project.

```
# Install Node.js LTS first
node --version
npm --version

# Create and run the app
npx create-expo-app@latest MyApp --template blank
cd MyApp
npx expo start

# Replace App.js with one practical at a time.
# For navigation/Firebase/Formik/AsyncStorage, install the packages
# listed in the relevant practical before running it.
```

2. Basic React Native App

Purpose: Displays text, an image, and a button alert.

```
import React from 'react';
import { View, Text, Image, Button, Alert } from 'react-native';

export default function App() {
  return <View style={{flex:1,justifyContent:'center',alignItems:'center',padding:20}}>
    <Text style={{fontSize:26,fontWeight:'bold'}}>Hello React Native</Text>
    <Text>My first mobile application</Text>
    <Image source={{uri:'https://reactnative.dev/img/tiny_logo.png'}}
      style={{width:100,height:100,margin:20}} />
    <Button title="Click Me" onPress={() => Alert.alert('Welcome!')} />
  </View>;
}
```

3. UI Design Using Core Components

Purpose: Shows reusable subject cards inside a scrollable screen.

```
import React from 'react';
import { ScrollView, View, Text } from 'react-native';

const subjects = ['React Native','JavaScript','Firebase','API Integration','Navigation'];

export default function App() {
  return <ScrollView contentContainerStyle={{padding:20}}>
    <Text style={{fontSize:26,fontWeight:'bold',marginBottom:16}}>React Native Subjects</Text>
    {subjects.map((name) => <View key={name}
      style={{padding:18,marginBottom:12,backgroundColor:'#eee',borderRadius:10}}>
      <Text style={{fontSize:18,fontWeight:'bold'}}>{name}</Text>
      <Text>Learn {name} fundamentals</Text>
    </View>)}
  </ScrollView>;
}
```

4. Styling Using StyleSheet and Flexbox

Purpose: Arranges boxes horizontally using Flexbox.

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return <View style={s.container}>
    <Text style={s.title}>Flexbox Layout</Text>
    <View style={s.row}>{[1,2,3].map(n =>
      <View key={n} style={s.box}><Text style={s.number}>{n}</Text></View>)}</View>
    <Text style={s.note}>Boxes use flexDirection: 'row'.</Text>
  </View>;
}
const s = StyleSheet.create({
  container:{flex:1,justifyContent:'center',padding:20,backgroundColor:'#f5f5f5'},
  title:{fontSize:26,fontWeight:'bold',textAlign:'center',marginBottom:25},
  row:{flexDirection:'row',justifyContent:'space-between',alignItems:'center'},
  box:{width:75,height:75,backgroundColor:'#2563eb',borderRadius:10,
    justifyContent:'center',alignItems:'center'},
  number:{fontSize:25,color:'white',fontWeight:'bold'},
  note:{marginTop:25,padding:16,backgroundColor:'white',textAlign:'center'}
});
```

5. User Input Form Creation

Purpose: Accepts name, email, and mobile number with basic validation.

```
import React, {useState} from 'react';
import {View,Text,TextInput,Button,Alert} from 'react-native';

export default function App() {
  const [name,setName]=useState(''), [email,setEmail]=useState(''), [mobile,setMobile]=useState('');
  const submit=()=>{
    if(!name.trim()||!email.trim()||mobile.length!==10)
      return Alert.alert('Error','Enter name, email and a 10-digit mobile number.');
    Alert.alert('Submitted',`${name}\n${email}\n${mobile}`);
    setName(''); setEmail(''); setMobile('');
  };
  const field=(label,value,set,props={})=><TextInput placeholder={label}
    value={value} onChangeText={set} style={{borderWidth:1,padding:12,marginVertical:7}}
    {...props}/>;
  return <View style={{flex:1,justifyContent:'center',padding:22}}>
    <Text style={{fontSize:24,fontWeight:'bold',marginBottom:12}}>Student Registration</Text>
    {field('Name',name,setName)}
    {field('Email',email,setEmail,{keyboardType:'email-address',autoCapitalize:'none'})}
    {field('10-digit mobile',mobile,setMobile,{keyboardType:'phone-pad',maxLength:10})}
    <Button title="Submit Form" onPress={submit}/>
  </View>;
}
```

6. Props and State in Functional Components

Purpose: Parent state is passed to a child through props and updated by a button.

```
import React,{useState} from 'react';
import {View,Text,Button} from 'react-native';

const Child=({message,name})=><View style={{padding:20}}>
  <Text>Child Component</Text><Text>{message}</Text><Text>Student: {name}</Text>
</View>;

export default function App() {
  const [message,setMessage]=useState('Hello from Parent');
  return <View style={{flex:1,justifyContent:'center',padding:24}}>
    <Child message={message} name="John"/>
    <Button title="Change Message" onPress={()=>setMessage('State updated!')}/>
  </View>;
}
```

7. Counter App Using State

Purpose: Increase, decrease, and reset a counter; the value cannot go below zero.

```
import React,{useState} from 'react';
import {View,Text,Button} from 'react-native';

export default function App() {
  const [count,setCount]=useState(0);
  return <View style={{flex:1,justifyContent:'center',alignItems:'center',gap:16}}>
    <Text style={{fontSize:28,fontWeight:'bold'}}>Counter App</Text>
    <Text style={{fontSize:52}}>{count}</Text>
    <View style={{flexDirection:'row',gap:12}}>
      <Button title="–" onPress={()=>setCount(n=>Math.max(0,n-1))}/>
      <Button title="+" onPress={()=>setCount(n=>n+1)}/>
      <Button title="Reset" onPress={()=>setCount(0)}/>
    </View>
  </View>;
}
```

8. Stack Navigation

Purpose: Navigates between Home and Details screens.

```
# Install:
# npm install @react-navigation/native @react-navigation/native-stack
# npx expo install react-native-screens react-native-safe-area-context

import React from 'react';
import {View,Text,Button} from 'react-native';
import {NavigationContainer} from '@react-navigation/native';
import {createNativeStackNavigator} from '@react-navigation/native-stack';

const Stack=createNativeStackNavigator();
const Home=({navigation})=><View style={{flex:1,justifyContent:'center',alignItems:'center'}}>
  <Text>Home Screen</Text><Button title="Go to Details" onPress={()=>navigation.navigate('Details')}/>
</View>;
const Details=({navigation})=><View style={{flex:1,justifyContent:'center',alignItems:'center'}}>
  <Text>Details Screen</Text><Button title="Go Back" onPress={()=>navigation.goBack()}/>
</View>;

export default function App() {
  return <NavigationContainer><Stack.Navigator>
    <Stack.Screen name="Home" component={Home}/>
    <Stack.Screen name="Details" component={Details}/>
  </Stack.Navigator></NavigationContainer>;
}
```

9. Bottom Tab Navigation

Purpose: Switches between Home, Profile, and Settings tabs.

```
# Install:
# npm install @react-navigation/native @react-navigation/bottom-tabs
# npx expo install react-native-screens react-native-safe-area-context

import React from 'react';
import {View,Text} from 'react-native';
import {NavigationContainer} from '@react-navigation/native';
import {createBottomTabNavigator} from '@react-navigation/bottom-tabs';

const Tab=createBottomTabNavigator();
const Screen=({name})=><View style={{flex:1,justifyContent:'center',alignItems:'center'}}>
  <Text style={{fontSize:24}}>{name} Screen</Text>
</View>;
const Home=()=> <Screen name="Home"/>;
const Profile=()=> <Screen name="Profile"/>;
const Settings=()=> <Screen name="Settings"/>;

export default function App() {
  return <NavigationContainer><Tab.Navigator>
    <Tab.Screen name="Home" component={Home}/>
    <Tab.Screen name="Profile" component={Profile}/>
    <Tab.Screen name="Settings" component={Settings}/>
  </Tab.Navigator></NavigationContainer>;
}
```

10. Passing Data Between Screens

Purpose: Sends student name and course from Home to Details using route params.

```
# Install stack navigation packages as in Practical 8.

import React,{useState} from 'react';
import {View,Text,TextInput,Button} from 'react-native';
import {NavigationContainer} from '@react-navigation/native';
import {createNativeStackNavigator} from '@react-navigation/native-stack';

const Stack=createNativeStackNavigator();
function Home({navigation}) {
  const [name,setName]=useState(''),[course,setCourse]=useState('');
  return <View style={{padding:20,gap:10}}>
    <TextInput placeholder="Student name" value={name} onChangeText={setName} style={{borderWidth:1,
padding:10}}/>
    <TextInput placeholder="Course" value={course} onChangeText={setCourse} style={{borderWidth:1,
padding:10}}/>
    <Button title="Open Details" onPress={()=>navigation.navigate('Details',{name:name||'Not provided',
course:course||'Not provided'})}/>
  </View>;
}
function Details({route}) {
  return <View style={{padding:20}}>
    <Text>Student: {route.params.name}</Text><Text>Course: {route.params.course}</Text>
  </View>;
}
export default function App() {
  return <NavigationContainer><Stack.Navigator>
    <Stack.Screen name="Home" component={Home}/><Stack.Screen name="Details" component={Details}/>
  </Stack.Navigator></NavigationContainer>;
}
```

11. Global State Using Context API

Purpose: Shares a user name between Profile and Dashboard without passing props.

```
import React,{createContext,useContext,useState} from 'react';
import {View,Text,TextInput,Button} from 'react-native';

const UserContext=createContext();
function Profile() {
  const {user,update}=useContext(UserContext),[name,setName]=useState('');
  return <View style={{padding:15}}>
    <Text>Profile: {user}</Text>
    <TextInput placeholder="New name" value={name} onChangeText={setName} style={{borderWidth:1,padding:
8}}/>
    <Button title="Update User" onPress={()=>{if(name.trim())update(name.trim());setName('')}}/>
  </View>;
}
function Dashboard() {
  const {user}=useContext(UserContext);
  return <Text style={{padding:15}}>Current user: {user}</Text>;
}
export default function App() {
  const [user,setUser]=useState('John');
  return <UserContext.Provider value={{user,update:setUser}}>
    <View style={{flex:1,justifyContent:'center'}}><Profile/><Dashboard/></View>
  </UserContext.Provider>;
}
```

12. API Integration Using Fetch

Purpose: Fetches posts, displays loading/error states, and supports retry.

```
import React,{useEffect,useState} from 'react';
import {View,Text,FlatList,ActivityIndicator,Button} from 'react-native';

export default function App() {
  const [posts,setPosts]=useState([]),[loading,setLoading]=useState(true),[error,setError]=useState('');
  const load=async()=>{
    setLoading(true);setError('');
    try {
      const r=await fetch('https://jsonplaceholder.typicode.com/posts');
      if(!r.ok) throw new Error('Request failed');
      setPosts((await r.json()).slice(0,15));
    } catch(e) {setError(e.message)}
    finally {setLoading(false)}
  };
  useEffect(()=>{load()},[]);
  if(loading)return <ActivityIndicator style={{flex:1}} size="large"/>;
  if(error)return <View style={{padding:20}}><Text>{error}</Text><Button title="Try Again" onPress={
load}/></View>;
  return <FlatList data={posts} keyExtractor={p=>String(p.id)}
    contentContainerStyle={{padding:16}}
    renderItem={({item})=><View style={{padding:12,marginBottom:10,backgroundColor:'#eee'}}>
      <Text style={{fontWeight:'bold'}}>{item.id}. {item.title}</Text><Text>{item.body}</Text>
    </View>}/>;
}
```

13. Form Validation Using Formik

Purpose: Validates name, email, and password before submitting.

```
# Install:
# npm install formik yup

import React from 'react';
import {View,Text,TextInput,Button,Alert} from 'react-native';
import {Formik} from 'formik';
import * as Yup from 'yup';

const schema=Yup.object({
  name:Yup.string().required('Name required'),
  email:Yup.string().email('Invalid email').required('Email required'),
  password:Yup.string().min(6,'Use at least 6 characters').required('Password required')
});
export default function App() {
  return <Formik initialValues={{name:'',email:'',password:''}} validationSchema={schema}
    onSubmit={v=>Alert.alert('Success',`Welcome ${v.name}`)}>
    {({handleChange,handleBlur,handleSubmit,values,errors,touched})=><View style={{padding:20,gap:8}}>
      {['name','email','password'].map(k=><View key={k}>
        <TextInput placeholder={k} secureTextEntry={k==='password'} value={values[k]}
          onChangeText={handleChange(k)} onBlur={handleBlur(k)} style={{borderWidth:1,padding:10}}/>
        {touched[k]&&errors[k]?<Text style={{color:'red'}}>{errors[k]}</Text>:null}
      </View>)}
      <Button title="Register" onPress={handleSubmit}/>
    </View>}
  </Formik>;
}
```

14. Firebase Email and Password Authentication

Purpose: Registers, signs in, and signs out users with Firebase Authentication.

```
# Install:
# npm install firebase
# Create a Firebase project and enable Authentication > Email/Password.
# Put your project's web configuration in firebaseConfig.js.

# firebaseConfig.js
import {initializeApp,getApp,getApps} from 'firebase/app';
import {getAuth} from 'firebase/auth';
const config={apiKey:'YOUR_API_KEY',authDomain:'YOUR_DOMAIN',
  projectId:'YOUR_PROJECT_ID',appId:'YOUR_APP_ID'};
export const auth=getAuth(getApps().length?getApp():initializeApp(config));

# App.js
import React,{useState} from 'react';
import {View,Text,TextInput,Button,Alert} from 'react-native';
import {createUserWithEmailAndPassword,signInWithEmailAndPassword,signOut}
  from 'firebase/auth';
import {auth} from './firebaseConfig';

export default function App() {
  const [email,setEmail]=useState(''),[password,setPassword]=useState(''),[user,setUser]=useState(null);
  const run=async(fn)=>{
    try {const r=await fn(auth,email,password);setUser(r.user.email)}
    catch(e){Alert.alert('Authentication error',e.message)}
  };
  return <View style={{flex:1,justifyContent:'center',padding:20,gap:10}}>
    <Text>{user?`Signed in: ${user}`:'Email Authentication'}</Text>
    <TextInput placeholder="Email" value={email} onChangeText={setEmail}
      autoCapitalize="none" keyboardType="email-address" style={{borderWidth:1,padding:10}}/>
    <TextInput placeholder="Password" value={password} onChangeText={setPassword}
      secureTextEntry style={{borderWidth:1,padding:10}}/>
    <Button title="Register" onPress={()=>run(createUserWithEmailAndPassword)}/>
    <Button title="Login" onPress={()=>run(signInWithEmailAndPassword)}/>
    <Button title="Logout" onPress={()=>signOut(auth).then(()=>setUser(null))}/>
  </View>;
}
```

15. AsyncStorage for Local Storage

Purpose: Saves and loads a profile name locally on the device.

```
# Install:
# npx expo install @react-native-async-storage/async-storage

import React,{useState} from 'react';
import {View,Text,TextInput,Button,Alert} from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';

export default function App() {
  const [name,setName]=useState(''),[saved,setSaved]=useState('');
  const save=async()=>{
    if(!name.trim())return Alert.alert('Enter a name');
    await AsyncStorage.setItem('profileName',name.trim());
    setSaved(name.trim());setName('');Alert.alert('Saved locally');
  };
  const load=async()=>setSaved(await AsyncStorage.getItem('profileName')||'No saved name');
  const remove=async()=>{await AsyncStorage.removeItem('profileName');setSaved('');};
  return <View style={{flex:1,justifyContent:'center',padding:20,gap:12}}>
    <Text style={{fontSize:24,fontWeight:'bold'}}>Local Profile</Text>
    <TextInput placeholder="Enter name" value={name} onChangeText={setName}
      style={{borderWidth:1,padding:12}}/>
    <Button title="Save Name" onPress={save}/>
    <Button title="Load Name" onPress={load}/>
    <Button title="Remove Name" onPress={remove}/>
    <Text>Saved name: {saved}</Text>
  </View>;
}
```